INFOR, Vol. 50, No. 2, May 2012, pp. 59–71
ISSN 0315-5986 | EISSN 1916-0615

The Vehicle Routing Problem with Stochastic Demands and Split Deliveries

Hongtao Lei
College of Information Systems and Management, National University of Defense Technology, Changsha, Hunan, China P.R., 410073, e-mail: hongtaolei@yahoo.cn

Gilbert Laporte
Canada Research Chair in Distribution Management and CIRRELT, HEC Montréal, 3000 chemin de la Côte-Sainte-Catherine, Montreal, Canada, H3T 2A7, e-mail: gilbert.laporte@cirrelt.ca

Bo Guo
College of Information Systems and Management, National University of Defense Technology, Changsha, Hunan, China P.R., 410073, e-mail: boguo@nudt.edu.cn

Abstract—This paper proposes a paired vehicle recourse policy for the Vehicle Routing Problem with Stochastic Demands and Split Deliveries (VRPSDSD). Most publications on the stochastic vehicle routing problem focus on recourse policies which assume that the vehicles operate independently of each other. An adaptive large neighborhood search heuristic is developed for this problem. Extensive computational experiments demonstrate that allowing split deliveries is sometimes beneficial, namely when expected demands lie between 51% and 70% of the vehicle capacity.

Keywords Stochastic vehicle routing, split deliveries, adaptive large neighborhood search metaheuristic.

1. INTRODUCTION

This paper considers the *Vehicle Routing Problem with Stochastic Demands and Split Deliveries* (VRPSDSD) defined on an undirected graph $G = (V, E)$, where $V = \{v_0, v_1, \ldots, v_n\}$ is the vertex set and $E = \{(v_i, v_j) : v_i, v_j \in V, i < j\}$ is the edge set. Vertex v_0 is a depot, while the remaining vertices represent customers. The number of vehicles is not limited and all vehicle have the same capacity Q. A symmetric travel cost matrix $C = (c(v_i, v_j))$ satisfying the triangle inequality is defined on E. With each vertex v_i is associated a nonnegative and stochastic demand ξ_i to be delivered. The demands can be split and are unknown until vehicles arrive at the vertices. If demands were unsplittable and deterministic, the aim of a Vehicle Routing Problem (VRP) defined on G would be to construct a set of feasible and least cost vehicle routes starting and ending at the depot and visiting each customer exactly once. However, in the VRPSDSD, the presence of splittable demands means that some vertices can belong to more than one vehicle route. In addition, the presence of stochastic demands means that routes cannot always be followed as planned and the solution cost must be minimized in the expected sense, as we will now explain.

The VRPSDSD can be modeled and solved as a stochastic mathematical program. The most common solution methodology for this class of problems is called *a priori optimization*, a concept initially proposed by Bertsimas et al. (1990) and applied by several authors to the field of vehicle routing (e.g. Bertsimas (1992), Laporte and Louveaux (1993), Gendreau et al. (1995), Gendreau et al. (1996), Laporte et al. (2002), Tan et al. (2007), Mendoza et al. (2010), Laporte et al. (2010) and Lei et al. (2011)). A priori optimization usually yields stable and practically predictable solutions (Bertsimas, 1992). In a priori optimization, a *first-stage solution* consisting of a set of planned vehicle routes is first constructed, and the realizations of the random variables are then revealed. Whenever the vehicle becomes empty along its route, *failure* is said to occur and a *recourse* action generating an extra cost is implemented in a *second-stage solution*. The aim of the VRPSDSD is to design a first-stage solution yielding a second-stage solution of least expected cost.

Received October 2011; Revision January–April 2012; Accepted April 2012

There exist many possible policies for the recourse action. Dror and Trudeau (1986) provide an expected cost objective function where all remaining customers are served by individual direct deliveries from the depot after a route failure. Dror et al. (1989) introduce the *traditional recourse* policy in which vehicles return to the depot to reload when a customer demand cannot be satisfied and resume their route at the vertex of failure. Dror (1993) analyzes the stochastic routing problem with recourse for a maximum of one failure per route. Gendreau et al. (1996) combine two types of uncertainty: stochastic demands and stochastic customers. These authors apply two types of recourse: in the first type the a priori routes are followed as planned, except that any absent customer is skipped; a second type is similar to the traditional recourse. Laporte et al. (2002) use the same recourse as in Dror et al. (1989) and consider multiple route failures with Poisson and normal demands. Some papers also consider the *preventive recourse* policy that uses a priori routes but the vehicle may preventively return to the depot to replenish before a failure occurs. Bertsimas et al. (1995), Yang et al. (2000) and Bianchi et al. (2004) use dynamic programming to construct routes that include preventive returns to the depot and improve the expected cost of the solutions. Novoa et al. (2006) present a set partitioning model for the VRP with stochastic demands. Ak and Erera (2007) were the first to provide a *paired-vehicle recourse* policy where some a priori vehicle routes are paired to pool the capacity of two vehicles in an attempt to reduce routing costs. In this policy, if a first vehicle becomes empty, the second vehicle which serves a lower amount of demand adds the unserved customers of the first vehicle to the end of its routes. The traditional recourse policy is used for the second vehicle when the vehicle is unable to serve the customers on its route. Chepuri and Homem-de Mello (2005) have developed a cross-entropy heuristic for the VRP with stochastic demands, whereas Goodson et al. (2012) have developed cyclic-order neighborhoods for the same problem. A survey of stochastic vehicle routing can be found in Cordeau et al. (2007).

Another strand of literature related to our recourse policy is the *Split Delivery Vehicle Routing Problem* (SDVRP) which allows customers to be visited more than once (split deliveries), in contrast to what is usually assumed in the classical VRP. The SDVRP was introduced by Dror and Trudeau (1989, 1990) who motivated their study by showing that split deliveries can yield savings. Dror et al. (1994) present valid inequalities for SDVRP and develop a constraint relaxation branch-and-bound algorithm. Belenguer et al. (2000) provide a better lower bound for the SDVRP in which customer demands do not exceed the vehicle capacity. Ho and Haugland (2004) provide a tabu search heuristic for the vehicle routing problem with time windows and split deliveries. Archetti et al. (2006) analyze the maximum possible savings obtained by allowing split deliveries, and Archetti et al. (2008) present a computational study to show how the savings of SDVRP depend on the characteristics of the instance. Nowak et al. (2008) quantify the benefit of using split loads for the pickup and delivery problem.

To our knowledge, this paper is the most comprehensive study to date on the VRP with stochastic demands and split deliveries. It is the first to propose an efficient heuristic to this problem. The remainder of the paper is organized as follows. Section 2 describes the recourse policy. In Section 3, we provide the detailed computation of the expected cost of solution. An adaptive large neighborhood search metaheuristic for the problem is described in Section 4, followed by extensive computational experiments in Section 5, and by conclusions in Section 6.

2. DESCRIPTION OF THE RECOURSE POLICY

In the paired vehicles recourse policy with split delivery, each vertex is initially assigned to one or two routes. Some routes may be matched together to create a route pair, but no route is included in more than one route pair. In unpaired routes, each vertex is served by only one vehicle. In paired routes, vertices can be served by one or two vehicles, and customer demand can be split into two predetermined percentages. No vertex is shared among more than two vehicles and any pair of routes cannot have more than one vertex in common. There are therefore two vertex types in our problem: (1) unsplit vertices which are served by only one vehicle and (2) split vertices which are served by two paired vehicles.

When failure occurs at an unsplit vertex, we apply the traditional recourse policy: when the demand of the vertex exceeds the remaining load of vehicle, the vehicle delivers its load, returns to the depot to reload, and then returns to the last visited vertex; when the demand is exactly equal to the remaining load of vehicle, the vehicle returns to the depot after visiting the current vertex and then proceeds to the next vertex. When failure occurs at a split vertex, each of the two paired vehicles assigned to this vertex serves its planned percentage of the vertex demand defined in the first stage solution.

We assume there can be only one split vertex on two paired routes, a result of the following proposition by Dror and Trudeau (1990) for the deterministic Split Delivery Vehicle Routing Problem (SDVRP). This proposition has not been proven for the stochastic case. However, it makes sense to use it in a heuristic context since it allows a remarkable reduction of the number of solutions of the search space.

Proposition 1. *If the costs $c(v_i,v_j)$ satisfy the triangle inequality, then there exists an optimal solution to the deterministic SDVRP in which no two routes have more than one customer with a split delivery in common.*

We make the following standard assumptions (Dror, 1993; Laporte et al., 2002, 2010; Lei et al., 2011) for the problem:

Assumption 1: The vertex demands ξ_i are independent random variables with known discrete or continuous probability distributions.

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

Assumption 2: Demands never exceed the vehicle capacity: ξ_i is a nonnegative random variable such that $P\{\xi_i \leq Q\} \cong 1$.

Assumption 3: No planned route r_k can have more than one failure on any route r_k: $P\{\sum_{v_i \in r_k} \xi_i \leq 2Q\} \cong 1$. Note that if the route is the one of the paired routes, the value of the demand of the split vertex in the formula should be the planned percentage of demand of the vertex on the route, denoted as $\alpha_i \xi_i$, where $0 < \alpha_i < 1$.

3. COMPUTATION OF THE EXPECTED COST OF A SOLUTION

We now show how to compute the expected cost of a solution under a standard non-cooperative recourse policy, and we discuss an extension to this policy to the cooperative case.

3.1. Definition of the total expected cost

Consider the k^{th} planned route $r_k = (v_0^k = v_0, v_1^k, v_2^k, \ldots, v_{n_k}^k, v_{n_k+1}^k = v_0)(k = 1, \ldots, m)$. Given a first-stage solution $R = (r_1, \ldots, r_m)$, the objective function $F(R)$ is equal to

$$F(R) = \phi(R) + E[\psi(R)], \tag{1}$$

where $\phi(R) = \sum_{k=1}^{m} \phi(r_k) = \sum_{k=1}^{m} \sum_{i=0}^{n_k} c(v_i^k, v_{i+1}^k)$ is the deterministic cost of the planned routes and $E[\psi(R)]$ is the expected cost of recourse. Its computation is detailed in the remainder of this section.

3.2. The expected cost of recourse

We consider the two types of failure at the vertices: (1) the failure that occurs when the demand of the vertex exceeds the remaining load of the vehicle; and (2) the failure that occurs when the demand of the vertex is exactly equal to the remaining load of the vehicle. Under the non-cooperative recourse policy, if two vehicles share a vertex, each operates independently of the other. In case of failure each vehicle is only responsible for its share of the demand of the split vertex defined in the first-stage solution. In other words, the split vertex can be seen as two unsplit vertices.

Proposition 2. *If failure happens when the vehicle visits vertex v_i^k on route r_k, the cost of recourse routes can be obtained according to two failure types as follows*:

(1) $s_i^k = 2c(v_i^k, v_0^k)$, *if the failure is of the first type*.
(2) $\bar{s}_i^k = c(v_i^k, v_0^k) + c(v_0^k, v_{i+1}^k) - c(v_i^k, v_{i+1}^k)$, *if the failure is of the second type*.

Proof. In case 1, s_i^k is the cost of a return trip between vertex v_i^k and the depot. In case 2, $\bar{s}_i^k$ is the cost of cost of going from v_i^k to the depot and then to v_{i+1}^k, minus the saving obtained by not-traveling from v_i^k to v_{i+1}^k. □

Let ξ_i^k be the demand of vertex v_i^k on route r_k. X_i^k is the cumulative served demand of vehicle up to v_i^k on the route r_k. We use two cases to model the demand. First, in the discrete case, let $P_D(t) = P\{D = t\}$ denote the probability function of a discrete random variable D. The probability $P_{\xi_i^k}$ is known for each vertex v_i^k of route r_k. Second, in the continuous case, let $f_D(t)$ denote the density function of a continuous random variable D. The probability $f_{\xi_i^k}$ is known for each vertex v_i^k of route r_k.

Proposition 3. *The cumulative demand X_i^k probability up to the vertex v_i^k on a route r_k can be computed by the following recursions*:

$$P_{X_i^k}(t) = \begin{cases} \sum_{l=0}^{t} P_{X_{i-1}^k}(t-l) P_{\xi_i^k}(l) & \text{if } v_i^k \text{ is an unsplit vertex}, \\ \sum_{l=0}^{t} P_{X_{i-1}^k}(t-l) P_{\alpha_i^k \xi_i^k}(l) & \text{if } v_i^k \text{ is a split vertex}, \end{cases} \tag{2}$$

with the boundary condition

$$P_{X_1^k}(t) = \begin{cases} P_{\xi_1^k}(t) & \text{if } v_1^k \text{ is an unsplit vertex}, \\ P_{\alpha_1^k \xi_1^k}(t) & \text{if } v_1^k \text{ is a split vertex}. \end{cases}$$

$$f_{X_i^k}(t) = \begin{cases} \int_{l=0}^{t} f_{X_{i-1}^k}(t-l) f_{\xi_i^k}(l) dl & \text{if } v_i^k \text{ is an unsplit vertex}, \\ \int_{l=0}^{t} f_{X_{i-1}^k}(t-l) f_{\alpha_i^k \xi_i^k}(l) dl & \text{if } v_i^k \text{ is a split vertex}, \end{cases} \tag{3}$$

with the boundary condition

$$f_{X_1^k}(t) = \begin{cases} f_{\xi_1^k}(t) & \text{if } v_1^k \text{ is an unsplit vertex}, \\ f_{\alpha_1^k \xi_1^k}(t) & \text{if } v_1^k \text{ is a split vertex}, \end{cases}$$

where α_i^k is the planned percentage of the demand for the split vertex v_i^k on route r_k, and $0 < \alpha_i^k < 1$.

Proof. The proof is similar to that presented in Lei et al. (2011) for the vehicle routing problem with stochastic demands and time windows. The main difference is that here unsplit and split vertices are handled separately. □

Proposition 4. *The probability P_i^k of route failure at v_i^k on a route r_k can be calculated as*:

$$P_i^k = \sum_{t=0}^{Q-1} P_{X_{i-1}^k}(t) - \sum_{t=0}^{Q-1} P_{X_i^k}(t), \tag{4}$$

in the discrete case and

$$P_i^k = \int_{t=0}^{Q} f_{X_{i-1}^k}(t) dt - \int_{t=0}^{Q} f_{X_i^k}(t) dt, \tag{5}$$

in the continuous case.

Proof. The proof is the same as that presented in Lei et al. (2011) for the vehicle routing problem with stochastic demands and time windows. □

The probability of failure of the second type at v_i^k can be calculated directly by the formulas of Proposition 3, as $\sum_{l=1}^{Q} P_{X_{i-1}^k}(Q-l) P_{\xi_i^k}(l)$ or $\int_{l=0}^{Q} f_{X_{i-1}^k}(Q-l) f_{\xi_i^k}(l) dl$ if v_i^k is an unsplit vertex, and as $\sum_{l=1}^{Q} P_{X_{i-1}^k}(Q-l) P_{\alpha_i^k \xi_i^k}(l)$ or $\int_{l=0}^{Q} f_{X_{i-1}^k}(Q-l) f_{\alpha_i^k \xi_i^k}(l) dl$ if v_i^k is a split vertex. The probability

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

of failure of the first type at v_i^k is equal to P_i^k, minus the probability of the failure of the second type.

Proposition 5. *If the probability distribution of vertex demand is discrete, then the expected cost of recourse for route r_k is*

$$E[\psi(r_k)] = \sum_{i=2}^{n_k} (\iota_i^k s_i^k + \kappa_i^k \bar{s}_i^k), \tag{6}$$

where ι_i^k is the probability that a failure of the first type occurs, and κ_i^k is the probability that a failure of the second type occurs. Using Propositions 3 and 4, the two terms can be computed as

$$\kappa_i^k = \begin{cases} \sum_{l=1}^{Q} P_{\xi_i^k}(l) P_{X_{i-1}^k}(Q-l) & \text{if } v_i^k \text{ is an unsplit vertex,} \\ \sum_{l=1}^{Q} P_{\alpha_i^k \xi_i^k}(l) P_{X_{i-1}^k}(Q-l) & \text{if } v_i^k \text{ is a split vertex,} \end{cases} \tag{7}$$

and

$$\iota_i^k = \sum_{t=0}^{Q-1} P_{X_{i-1}^k}(t) - \sum_{t=0}^{Q-1} P_{X_i^k}(t) - \kappa_i^k. \tag{8}$$

Proof. To evaluate the expected cost of recourse for r_k, we again distinguish between the two failure types and the two recourse actions these induce. Regarding the first failure type, s_i^k is the cost of recourse defined in Proposition 2, and the expected cost of recourse action at v_i^k is the cost s_i^k of recourse multiplied by the probability ι_i^k of failure of the first type. Regarding the second failure type, $\bar{s}_i^k$ is the cost of recourse, and the expected cost of recourse action at v_i^k in this case is the cost $\bar{s}_i^k$ of recourse multiplied by the probability κ_i^k of failure of the second type. Note that, in this case, no failure occurs if the demand of the last vertex in a route is exactly equal to the remaining load of the vehicle.

The total expected cost of recourse at v_i^k is the sum of the expected costs of recourse under the two failure types. Since $P\{\xi_i \le Q\} \cong 1$, the expected cost of recourse for r_k is the sum of the expected costs of recourse action at all the vertices after the first vertex on r_k. □

The expected cost of recourse of a solution is

$$E[\psi(R)] = \sum_{k=1}^{m} E[\psi(r_k)]. \tag{9}$$

We can extend Proposition 5 to the continuous case. In order to compute $E[\psi(r_k)]$, we adapt Proposition 4 to the continuous case. The convolution $\int_{l=0}^{Q} f_{\xi_i^k}(Q-l) f_{X_{i-1}^k}(l)dl$ is the probability of failure of the second type at v_i^k if v_i^k is an unsplit vertex, and $\int_{l=0}^{Q} f_{\alpha_i^k \xi_i^k}(Q-l) f_{X_{i-1}^k}(l)dl$ is the probability of failure of the second type at v_i^k if v_i^k is a split vertex. Also, $\int_{t=0}^{Q} f_{X_{i-1}^k}(t)dt - \int_{t=0}^{Q} f_{X_i^k}(t) - \int_{l=0}^{Q} f_{\xi_i^k}(Q-l) f_{X_{i-1}^k}(l)dl$ and $\int_{t=0}^{Q} f_{X_{i-1}^k}(t)dt - \int_{t=0}^{Q} f_{X_i^k}(t) - \int_{l=0}^{Q} f_{\alpha_i^k \xi_i^k}(Q-l) f_{X_{i-1}^k}(l)dl$ give the probability of the failure of the first type at v_i^k, according to the two types of v_i^k.

3.3. Note on the cooperative case

When failure occurs at the split vertex, some cooperation between the two vehicles involved in the split delivery can also be considered in order to reduce the expected cost of recourse. Under the cooperative recourse policy, the recourse action is only modified for a failure of the first type occurring at the split vertex. It is applied according to the arrival times of the two vehicles at the split vertex and to the state of each vehicle on its route. If the vehicle arriving second has not already failed on its route, the first vehicle will deliver as much demand as possible at the split vertex (usually more than the planned demand) when it resumes deliveries upon returning from the depot after a failure of the first type; the amount of demand serviced is computed in such a way that the second vehicle will not fail twice on the remaining part of its route. If the second vehicle has already failed, the first vehicle will deliver its planned demand of the split vertex since the second vehicle cannot fail a second time. For a failure of the second type, since the vehicle delivers exactly the planned demand, it is not necessary to apply the cooperative policy.

Assume two paired routes r_{k_1} and r_{k_2} share a common split vertex, and let the split vertex be the i^{th} vertex on route r_{k_1} and the j^{th} vertex on route r_{k_2} ($v_i^{k_1} = v_j^{k_2}$). In case of the failure at the split vertex on route r_{k_1}, we divide the possible recourse actions into five cases as shown in Table 1, where $V1$ is the vehicle on route r_{k_1}, and $V2$ is the vehicle on route r_{k_2}. "State of $V2$" specifies whether $V2$ has already failed before arriving at the split vertex.

TABLE 1.
Cases when $V1$ fails at split vertex under a cooperative recourse policy

Case	Arrival order of $V1$	State of $V2$	Recourse action of $V1$	Demand served by $V1$
1	first	failed	non-cooperative recourse	planned percentage
2	first	not failed	cooperative recourse for failure of type 1 non-cooperative recourse for failure of type 2	planned percentage and perhaps more
3	second	failed	non-cooperative recourse	remaining percentage (as planned or less)
4	second	not failed	non-cooperative recourse	planned percentage

ISSN 0315-5986 | EISSN 1916-0615
DOI 10.3138/infor.50.2.059

With respect to case 1, because $V1$ is the first arriving vehicle at the split vertex and $V2$ has failed on its route, $V1$ applies the non-cooperative recourse policy and delivers its planned percentage of the demand when it fails at the split vertex. In case 4, since $V2$ is the first arriving vehicle and has successfully delivered its planned percentage of the demand at the split vertex, when $V1$ fails at that vertex, $V1$ applies the non-cooperative recourse policy and delivers the planned amount of demand. In case 2, if $V1$ experiences a failure of the first type at the split vertex, it will deliver as much demand as possible to reduce the likelihood of $V2$ failing on its route; if $V1$ experiences a failure of the second type, it will apply the non-cooperative recourse policy and deliver exactly its planned percentage of the demand. In case 3, $V2$ will apply the cooperative recourse policy when failing at the split vertex, and $V1$ will apply the non-cooperative policy.

Although the cooperative recourse policy is in principle superior to the non-cooperative policy, because it decreases the expected cost of failure of the second arriving vehicle at the split vertex, it does not have much impact on the expected solution costs from an empirical point of view. We have implemented and tested it but it did not result in significant savings, probably because the occurrence of case 2 to which cooperative recourse applies is rare. Indeed cooperation can only occur at a split vertex when $V2$ has not already failed. For the time being, its interest remains mostly conceptual.

4. ADAPTIVE LARGE NEIGHBORHOOD SEARCH HEURISTIC

We have developed an adaptive large neighborhood search heuristic for the VRPSDSD. This type of heuristic was introduced by Ropke and Pisinger (2006) in the context of the pickup and delivery vehicle problem with time windows. It was recently applied by Laporte et al. (2010) to the capacitated arc routing problem with stochastic demands and by Lei et al. (2011) to the capacitated vehicle routing with stochastic demands and time windows. This metaheuristic must be fine tuned to each application. In this section we describe its application to the VRPSDSD.

We obtain an initial solution by means of a construction heuristic which includes split deliveries and route pairings. As in Shaw (1997), at each iteration, q vertices are removed from their route by using one of several removal operators, and are reinserted by means of one of several insertion operators, where q is randomly selected in the interval $[\lceil 0.1n \rceil, \lceil 0.2n \rceil]$ as Laporte et al. (2010). Our worst removal and split insertion operators are new, and the others are inspired from past research (Shaw, 1998; Ropke and Pisinger, 2006; Laporte et al., 2010). At each iteration, the removal and insertion operators are selected according to a probabilistic mechanism. Note that, the insertion procedures during the construction and four insertion operators are implemented without violating Assumption 3. In practice, the feasibility of a potential insertion is first tested with a threshold value: if Y^k is the total demand of route r_k after the insertion of the vertex,then the insertion is deemed feasible if $P\{Y^k \leq 2Q\} > 0.9$.

4.1. Formal summary of the algorithm

Our implementation of the adaptive large neighborhood search heuristic can be formally summarized as follows.

Step 1: Initialize the parameters and construct a first solution. Set the objective value of the initial solution as the *record* and the *best cost*, compute the *deviation*. Set the initial solution as the *best solution* and define it as the *current solution*.

Step 2: Select a removal operator and an insertion operator based on the weights of the current iteration and move to a new solution by applying these operators.

Step 3: If the objective value of the new solution is smaller than the *best cost*, set the new solution as the *best solution* and set its objective value as the *best cost*. If the new solution is accepted using the RRT criterion, set it as the *current solution*. If the objective value of the new solution is smaller than the *record*, update the *record* and the *deviation*.

Step 4: Update the scores and the frequencies of the selected operators. Update the weights of all removal and insertion operators. Every ϱ iterations, reset their scores and frequencies.

Step 5: If the stopping criterion is met, output the *best solution* and the *best cost*. Otherwise go to Step 2.

4.2. Construction of an initial solution

We have devised the following construction heuristic to generate an initial solution. The heuristic first sorts the vertices in increasing order of their expected demand. Once the vertices are sorted, starting from the first vertex the heuristic sequentially selects the cheapest insertion over all routes r_k without violating Assumption 3. In order to generate the initial solution efficiently, the insertion cost computation only considers the cost associated with the deterministic part of the route. The insertion cost of vertex v_u in position i of route r_k is denoted by $I_{v_u}(i, r_k)$ and given by

$$I_{v_u}(i, r_k) = \Delta_i \phi(r_k), \tag{10}$$

where $\Delta_i \phi(r_k) = c(v^k_{i-1}, v^k_u) + c(v^k_u, v^k_i) - c(v^k_{i-1}, v^k_i)$.

If there is no feasible insertion for the current vertex, we sequentially search the routes in the current solution for the a first suitable single route (unpaired) where a fraction of the demand of the vertex can be inserted into the position with the least insertion cost. This fraction is the largest value that can be inserted into the route without violating Assumption 3. For simplicity, we consider a limited number of values of this percentage with an initial value of 0.1 and an increment of 0.1. If a first feasible single route exists, the remaining fraction of the vertex demand will be inserted into a second route which will then be paired with the first one. If no such feasible

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

second route exists, a new route is then created and the current vertex is inserted into the new route without split. The heuristic stops when all vertices have been inserted.

4.3. Removal and insertion operators

We use four removal operators and four insertion operators for the removal-insertion operations. In the removal operators, if a removed vertex belongs to two routes, it is removed from each of them.

4.3.1. Similarity removal (SR)

This operator extends the heuristic proposed by Shaw (1998). The idea is to remove vertices that are similar to each other. We define a similarity measure between two vertices v_i and v_j as

$$S(i,j) = \frac{1}{c(v_i, v_j)/c_i^{max} + \tau_{ij}}, \tag{11}$$

where $c(v_i,v_j)$ is the travel cost from v_i to v_j, c_i^{max} is the largest of $c(v_i, v_l)$ or $c(v_l, v_i)$ for all vertices l that have not yet been removed, and τ_{ij} is equal to 1 if v_i and v_j are in different routes, and 0 otherwise.

The operator first randomly selects an initial vertex to add into an empty set of removal vertices. For each additional removal vertex, the selection procedure is made up of the following steps. First, a seed vertex is chosen randomly from the set of removal vertices. Second, all the remaining vertices are sorted in decreasing order of the value of the similarity measure to the seed vertex. Third, the first vertex among the remaining vertices is removed from the route and added into the set of the removal vertices. The operator stops when q vertices have been removed from the current solution.

4.3.2. Deterministic worst removal (DWR)

The purpose of this operator, inspired from Ropke and Pisinger (2006) and Laporte et al. (2010), is to remove vertices that appear to be inserted in the wrong position in the current solution. Only the cost of the first-stage solution is considered to limit the computational effort, but this cost is already a good saving indicator. Let $\Delta\phi(i,k) = \phi(r_k) - \phi_{-i}(r_k)$ be the cost difference, where $\phi(r_k)$ is the deterministic planned cost of route r_k, and $\phi_{-i}(r_k)$is the deterministic planned cost of the route without vertex v_i^k.

The operator first computes the cost difference associated with removal of one vertex from each route and finds the vertex with the largest cost difference in each route. It then considers two cases: (1) if the number of the routes in the current solution is larger than q, the heuristic sorts the routes in decreasing order of their largest vertex cost difference, removes the vertex with the largest difference value from the first q routes, and then stops; (2) otherwise, it iteratively removes the vertex with the largest value of $\Delta\phi(i,k)$ in each route until q vertices have been removed.

4.3.3. Recourse worst removal (RWR)

The RWR operator concentrates on "critical" vertices that appear to be placed in the wrong position. In contrast to DWR, the operator focuses on the value of the expected recourse cost of the vertices in each route. It first computes the expected recourse cost for the vertices. It then sorts the routes in decreasing order of their largest expected vertex recourse cost value. If the number of routes in the current solution is at least equal to q, the operator removes the vertex with the largest recourse cost value from the first q routes and then stops. Otherwise, it sequentially removes the vertex with the largest recourse cost value from each route of the current solution until q vertices have been removed. Note that the heuristic skips the routes with just one vertex, since the demand of that vertex is always smaller than the capacity of vehicle and failure on the route will never occur.

4.3.4. Random removal (RR)

As in Ropke and Pisinger (2006), this operator randomly selects q vertices and removes them from their route. Its purpose is to diversify the search.

4.3.5. Split insertion (SI)

The split insertion operator splits the demand of the removed vertex into two unpaired routes, according to the expected cumulative demands of the two routes. The operator first enumerates all possible sets of two unpaired single routes where one route delivers a percentage of the demand of the inserted vertex and the other route delivers the remaining demand, without violating Assumption 3. The percentages are defined as

$$\alpha_1 = \frac{\sum_{\gamma=1}^{n_2} E[\xi_\gamma^2]}{\sum_{\gamma=1}^{n_1} E[\xi_\gamma^1] + \sum_{\gamma=1}^{n_2} E[\xi_\gamma^2]} \text{ and } \alpha_2 = 1 - \alpha_1, \tag{12}$$

where n_1 is the number of vertices on the first single route and n_2 is the number of vertices on the second single route. Here, the splits of Section 4.2 are not considered.

For each possible route, the percentage of the demand is inserted into the position with the least increase in the total expected cost of the route. After computing the increased cost by reinserting the removed vertex into all the possible paired routes, the operator inserts the vertex in the two routes with the least increase in the total expected cost and then marks the two routes as paired routes. If there exist no two such routes, the operator generates a new route for the vertex and inserts it into the route without split.

4.3.6. Greedy insertion (GI)

The operator sequentially inserts the q removed vertices into the current solution with the least increased cost, like in Ropke and Pisinger (2006) and Laporte et al. (2010). Here, for the sake of computational efficiency, we only consider the effect on the deterministic planned term of the objective

DOI 10.3138/infor.50.2.059

function. Formally, let $I(i,r_k)$ denote the increased cost of inserting the vertex in position i of route r_k, while respecting Assumption 3. The operator chooses the best insertion position (i,r_k) for the removed vertices as

$$(i, r_k) := \arg \min_{1 \le i \le n_{k+1}, r_k \in R} I(i, r_k), \tag{13}$$

where n_k is the number of vertices on route r_k, and position n_{k+1} is located between the last vertex $v^k_{n_k}$ and the depot. We note that only one route is changed during this insertion process.

4.3.7. *Regret insertion (RI)*

The main characteristic of RI operator is to incorporate a look-ahead information when selecting the removed vertex to be inserted. Like in Potvin and Rousseau (1993), our RI operator also uses a regret criterion. It firstly computes the best position with the lowest increased cost of $\Delta\phi(r_k)$ in each route for the q removed vertices. The selected vertex v to be inserted is determined as

$$v := \arg \max_{v \in M} \left\{ \frac{1}{z} \sum_{j=1}^{z} (\Delta_v \phi(r_j) - \Delta_v \phi(r_*)) \right\}, \tag{14}$$

where M is the set of removed vertices, $\Delta_v\phi(r_*)$ is the lowest increased cost among all routes when inserting v and this lowest increased cost occurs when inserting v in route r_*, $\Delta_v\phi(r_j)$ is the lowest increased cost in route r_j (excluding r_*), and z is the number of the routes (excluding r_*) in which v can be inserted. All insertions respect Assumption 3.

After choosing v, we insert it into route r_* in the position minimizing the increased cost $\Delta_v\phi(r_*)$. Also, we give priority to the vertices that cannot be inserted into the existing routes of the current solution. In this case, we generate a new route and insert the vertex into it.

4.3.8. *Demand and failure sorting insertion (DFSI)*

Inspired from Laporte et al. (2010), this operator sorts the removed vertices in decreasing order of their expected demand, and sorts the routes in increasing order of the value of their probability of failure. It then iteratively selects the vertex with the largest expected demand from the removed vertices, and inserts it into the first route of the solution while respecting Assumption 3. The best inserting position is the one yielding the least expected total cost. If the vertex cannot be inserted into an existing route, a new route is created. After each insertion, the probability of failure of each route is recomputed.

4.4. Adaptive heuristic selection mechanism

A weight w_i is associated with each removal and insertion operator i. At each iteration, given h operators with weights w_i, operator j is chosen with probability $w_j / \sum_{i=1}^{h} w_i$. The weight of each operator is updated every ϱ iterations. In our implementation, we use $\varrho = 50$. Initially, all operators have a weight of 1. At the j^{th} sequence of ϱ iterations, w_{ij} is updated as

$$w_{i,j+1} = w_{ij}(1 - \chi) + \chi \pi_{ij} / \varepsilon_{ij}, \tag{15}$$

where ε_{ij} and π_{ij} are the number of times operator i has been used in the j^{th} sequence of ϱ iterations (its frequency), and the score of operator i in the j^{th} sequence of ϱ iterations, respectively, and $\chi \in [0, 1]$ is a parameter equal to 0.1 in our implementation.

As in Ropke and Pisinger (2006) and Laporte et al. (2010), the score π_{ij} of the operator i in the j^{th} sequence of ϱ iterations can be computed as

$$\pi_{ij}^{(j-1)\varrho+\gamma+1} = \pi_{ij}^{(j-1)\varrho+\gamma} + \begin{cases} 30 & \text{if the operator results in a new global best solution,} \\ 10 & \text{if the operator results in a new solution improved on the current solution,} \\ 6 & \text{if the operator results in an accepted solution but not improved upon the current solution,} \end{cases} \tag{16}$$

with $\gamma = 1, \ldots, \varrho - 1$ and the boundary condition $\pi_{ij}^{(j-1)\varrho+1} = 0$ at the start of the j^{th} sequence of ϱ iterations.

4.5. Solution acceptance and stopping criteria

The acceptance criterion for a new solution is based on the record-to-record travel (RRT) algorithm of Dueck (1993). Let f^* be the value of the best current solution, called a *record*, and δ a positive parameter called a *deviation* and equal to $0.01f^*$ in our implementation. Let R be a solution, R' a neighbor of R, and $f_{R'}$ the objective value of solution R'. Solution R' is accepted if $f_{R'} < f^* + \delta$, and f^* is updated if $f_{R'} < f^*$. The algorithm ends if solution quality has not improved for a given number of iterations or if a preset number of iterations have been executed. In our implementation, these values are equal to 300 and 1000, respectively.

5. COMPUTATIONAL EXPERIMENTS

The algorithm described in Section 4 was coded using Matlab 7.0.4 and run on a laptop with 2 GHz dual processor and 2 GB RAM. We now describe the results of extensive computational experiments.

We assume that all demands in the experiments are Poisson distributed. This means that the cumulative probability distribution can be computed directly. For example, if $\xi_i^k \sim \text{Poisson}(\mu_i^k)$ for the unsplit vertex v_i^k on route r_k, then $X_i^k = \sum_{j \le i} \xi_j^k \sim \text{Poisson}(\sum_{j \le i} \mu_j^k)$. For the split vertex v_i^k on route r_k, if $\xi_i^k \sim \text{Poisson}(\mu_i^k)$, then $\alpha_i^k \xi_i^k \sim \text{Poisson}(\alpha_i^k \mu_i^k)$ and $X_i^k = \alpha_i^k \xi_i^k + \sum_{j<i} \xi_j^k \sim \text{Poisson}(\alpha_i^k \mu_i^k + \sum_{j<i} \mu_j^k)$.

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

5.1. Experiments on the modified Solomon instances

We now describe a first set of experiments performed on the Solomon instances.

5.1.1. Experimental design

We have generated test instances derived from those of Solomon (1987). The coordinates of the vertices are the same as in the Solomon instances and the demand of each vertex was generated according to a Poisson distribution having a mean value equal to the vertex demand, while the time windows and service times are not used. We consider six classes of instances: R1, R2, C1, C2, RC1 and RC2. The coordinates of R1 and R2 are the same, and so are those of RC1 and RC2. However, the coordinates of C1 and C2 are not identical. Hence we consider four types of instances: R, C1, C2, and RC. All instances have 100 customer vertices, and we just consider the first 25 or 50 vertices in the tests. The vehicle capacities in the original instances were too large for our problem, which meant that the failure probabilities were too small. We have therefore reduced them to $Q = 70$ while ensuring that $P\{\xi_i \leq Q\} \cong 1$ for each vertex v_i.

Because there are no comparative data and no competing heuristic exists for our problem, comparisons with best known solutions are not possible. However, we can compare the solutions of the standard VRPSD (without split demands), with at most one failure per route, and the solutions obtained by our heuristic of Section 4. Because there is no available proven optimal solution for the VRPSD used in the comparison, we have adapted the heuristic described in Section 4 to obtain a good solution to this problem, by just removing the possibility of splits. The stopping criterion of the adapted heuristic is similar to that of the heuristic described in Section 4.

Furthermore, in order to study how the benefit obtained by our recourse policy varies, we also transform the mean demands to the interval $[lQ, uQ]$, where $l < u$ are given numbers in (0,1]. As in Ho and Haugland (2004), the new mean demands are defined as

$$E[\xi_i]' = lQ + \frac{(u-l)(E[\xi_i] - \underline{E[\xi_i]})}{\overline{E[\xi_i]} - \underline{E[\xi_i]}} Q, \qquad (17)$$

where $\overline{E[\xi_i]} = \max_i\{E[\xi_i]\}$ and $\underline{E[\xi_i]} = \min_i\{E[\xi_i]\}$. Four configurations of $[l, u]$ are considered in the computations: (1) [0.01, 0.25]; (2) [0.26,0.50]; (3) [0.51, 0.70]; (4) no change. However, for $\xi_i^k \sim \text{Poisson}(0.7Q)$, $P\{\xi_i^k \leq Q\} \approx 0.99$.

The detailed information of the modified Solomon instances tested is shown in Table 2.

TABLE 2.
Modified Solomon instances

No.	Type	$\lvert V\rvert$	Q	$[l,u]$
mS-1	R	25	70	[0.01,0.25]
mS-2	R	25	70	[0.26,0.50]
mS-3	R	25	70	[0.51,0.70]
mS-4	R	25	70	–
mS-5	R	50	70	[0.01,0.25]
mS-6	R	50	70	[0.26,0.50]
mS-7	R	50	70	[0.51,0.70]
mS-8	R	50	70	–
mS-9	C1	25	70	[0.01,0.25]
mS-10	C1	25	70	[0.26,0.50]
mS-11	C1	25	70	[0.51,0.70]
mS-12	C1	25	70	–
mS-13	C1	50	70	[0.01,0.25]
mS-14	C1	50	70	[0.26,0.50]
mS-15	C1	50	70	[0.51,0.70]
mS-16	C1	50	70	–
mS-17	C2	25	70	[0.01,0.25]
mS-18	C2	25	70	[0.26,0.50]
mS-19	C2	25	70	[0.51,0.70]
mS-20	C2	25	70	–
mS-21	C2	50	70	[0.01,0.25]
mS-22	C2	50	70	[0.26,0.50]
mS-23	C2	50	70	[0.51,0.70]
mS-24	C2	50	70	–
mS-25	RC	25	70	[0.01,0.25]
mS-26	RC	25	70	[0.26,0.50]
mS-27	RC	25	70	[0.51,0.70]
mS-28	RC	25	70	–
mS-29	RC	50	70	[0.01,0.25]
mS-30	RC	50	70	[0.26,0.50]
mS-31	RC	50	70	[0.51,0.70]
mS-32	RC	50	70	–

5.1.2. Computational results

Table 3 presents the computational results for the instances with different geographical types and various values of $[l,u]$. The column "Unsplit-VRPSD" summarizes the results obtained from the VRPSD solutions without splits and with a maximum one failure per route. The column "Search heuristic" summarizes the results obtained by applying the heuristic of Section 4. The columns "# Routes" and "Cost" describe the number of routes of the solutions and the total expected cost of the solutions. We also report the total CPU time in seconds in the "Seconds" columns for "Unsplit-VRPSD" and "Search heuristic". The "Imp(%)" column shows the percentage improvement in "Cost" obtained by our heuristic, compared with the unsplit VRPSD. Each instance is run five times and we choose the best solution.

Table 3 shows that the instances in the range of [0.51, 0.70] always yield clear improvements (between 2.61% and 11.21%), while the instances in the other ranges yield no benefit. Our results are consistent with those obtained on the deterministic split delivery problem (Ho and Haugland, 2004; Archetti et al., 2008; Nowak et al., 2008): the benefit obtained by allowing split deliveries varies according to the ratio between vertex demand and vehicle capacity; the largest benefit is obtained

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

TABLE 3.
Comparisons on the modified Solomon instances

				Unsplit-VRPSD			Search heuristic			
No.	Type	$\|V\|$	$[l,u]$	# Routes	Cost	Seconds	# Routes	Cost	Seconds	Imp(%)
mS-1	R	25	[0.01,0.25]	3	397.896	1304.891	3	397.896	1351.936	0
mS-2	R	25	[0.26,0.50]	8	778.551	572.703	8	778.551	316.063	0
mS-3	R	25	[0.51,0.70]	13	1231.680	137.609	16	1155.115	1935.156	6.23
mS-4	R	25	–	4	519.750	423.406	4	519.750	371.906	0
mS-5	R	50	[0.01,0.25]	4	694.747	4276.828	4	694.747	1634.984	0
mS-6	R	50	[0.26,0.50]	14	1566.079	1193.781	14	1566.079	4152.797	0
mS-7	R	50	[0.51,0.70]	25	2559.670	522.109	24	2492.748	3434.563	2.61
mS-8	R	50	–	8	1029.687	1427.844	8	1029.687	2537.547	0
mS-9	C1	25	[0.01,0.25]	3	251.800	747.484	3	251.800	614.594	0
mS-10	C1	25	[0.26,0.50]	8	643.964	357.031	8	643.964	386.453	0
mS-11	C1	25	[0.51,0.70]	13	1092.425	257.391	17	969.979	545.188	11.21
mS-12	C1	25	–	5	483.343	504.672	5	483.343	360.422	0
mS-13	C1	50	[0.01,0.25]	5	511.093	2580.438	5	511.093	7369.750	0
mS-14	C1	50	[0.26,0.50]	14	1375.777	621.328	14	1375.777	1227.672	0
mS-15	C1	50	[0.51,0.70]	25	2341.479	512.219	25	2265.759	734.172	3.23
mS-16	C1	50	–	9	1004.869	646.859	9	1004.869	1263.391	0
mS-17	C2	25	[0.01,0.25]	3	315.844	1707.438	3	315.844	1643.641	0
mS-18	C2	25	[0.26,0.50]	7	747.530	271.578	8	747.530	416.578	0
mS-19	C2	25	[0.51,0.70]	13	1246.963	226.484	13	1159.721	586.516	7.00
mS-20	C2	25	–	6	570.735	662.828	6	570.735	535.516	0
mS-21	C2	50	[0.01,0.25]	5	619.911	2271.906	5	619.911	5441.781	0
mS-22	C2	50	[0.26,0.50]	15	1554.918	812.484	15	1554.918	1598.547	0
mS-23	C2	50	[0.51,0.70]	25	2631.183	645.813	24	2547.591	2009.422	3.18
mS-24	C2	50	–	10	1125.263	1268.313	10	1125.263	1150.328	0
mS-25	RC	25	[0.01,0.25]	3	459.660	299.109	3	459.660	1079.078	0
mS-26	RC	25	[0.26,0.50]	8	1143.068	278.016	8	1143.068	906.016	0
mS-27	RC	25	[0.51,0.70]	13	1845.985	162.313	15	1658.388	515.344	10.16
mS-28	RC	25	–	6	877.690	489.313	6	877.691	266.172	0
mS-29	RC	50	[0.01,0.25]	5	840.318	913.266	5	840.318	2713.000	0
mS-30	RC	50	[0.26,0.50]	15	2277.816	1088.625	15	2277.816	806.469	0
mS-31	RC	50	[0.51,0.70]	25	3917.402	640.531	25	3686.300	3175.969	5.90
mS-32	RC	50	–	10	1641.481	672.984	10	1641.481	2015.641	0
Average				**10.313**	**1196.831**	**890.550**	**10.563**	**1167.731**	**1659.269**	**1.55**

when the mean demand is greater than half the vehicle capacity, and the benefits do not depend on the geographical locations of the customers. Also, the best split delivery solutions have more routes. This is not surprising given that the additional capacity yielded by these extra routes acts as a buffer against uncertainty.

We also note that on some instances for which the benefits are observed, the number of vehicle routes is larger than in the compared unsplit VRPSD problem, e.g. the instance of type R in [0.51, 0.70] with 25 vertices. Whereas in the deterministic split delivery problem cost reductions appear to be due primarily to the ability to reduce the number of routes (Archetti et al., 2008), this may be different in the stochastic case because the total solution cost in the stochastic routing problem includes not only the deterministic planned route cost, but also the expected cost of recourse. If the number of routes of the solution is reduced, the average filling rate of the vehicles is higher, which leads to higher failure probabilities and a larger expected cost of recourse.

Table 4 shows the total visit number of vertices in each instance. From Tables 3 and 4, we can see that when an improvement is obtained, the total number of visits also increases. This is because our recourse policy uses the split deliveries which result in savings.

We have also evaluated the performance of each insertion and removal operator described in Section 4 on the modified Solomon instances. Table 5 reports the weights associated with each operator. This weight is obtained as the mean value of the final weights used to get the results of Table 3. The best insertion operators are GI and RI, whereas the best removal operators are SR and RWR. Compared to the other insertion operators, GI and RI focus on the deterministic part of the objective function and usually yield good savings. We

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

TABLE 4.
Total number of vertex visits in the modified Solomon instances

$\|V\|$	$[l,u]$	R	C1	C2	RC
25	[0.01,0.25]	25	25	25	25
	[0.26,0.50]	25	25	25	25
	[0.51,0.70]	31	33	30	31
	–	25	25	25	25
50	[0.01,0.25]	50	50	50	50
	[0.26,0.50]	50	50	50	50
	[0.51,0.70]	58	58	57	57
	–	50	50	50	50

note that the weight of SI in [0.51,0.70] is significantly larger than that observed in the other ranges. This is because in this range the SI operator splits the larger demand into two routes, which reduces the probability of failure. We do not claim that our selection of operators is the best possible. This question is left as future work.

5.2. Experiments on the modified Christiansen and Lysgaard instances

We next present a second set of experiments performed on a second set of instances.

5.2.1. *Experimental design*

We have also generated test instances derived from those of Christiansen and Lysgaard (2007). Like in Christiansen and Lysgaard (2007), the instances chosen include all instances of the Augerat test sets A and P and the Christofides and Eilon test set with up to 60 customers. Demands are Poisson distributed and the customers expected demands are equal to their original deterministic demands; vertex coordinates are left unchanged and are deterministic. The vehicle capacities are reduced to $Q = 70$ for each instance and some Christiansen and Lysgaard instances are omitted because they violate Assumption 2. We have therefore only kept those instances satisfying $P\{\xi_i \leq Q\} \cong 1$ for each vertex v_i. In the tested instances, the number of vehicles is unlimited.

Because our problem is different from that of Christiansen and Lysgaard who make different assumptions from those of this paper and work with a different recourse policy, a direct comparison with the results of these authors is again impossible. But as in the experiments on the modified Solomon instances, we can compare the solutions of the standard VRPSD and the solutions generated by the initial construction heuristic of Section 4.2, with those obtained by the heuristic of Section 4. Since split deliveries only yield a benefit when customer demand is in the interval $[0.51Q,0.70Q]$, the tests on the modified Christiansen and Lysgaard instances were only performed for this case.

The detailed information of the modified Christiansen and Lysgaard instances tested is shown in Table 6.

TABLE 6.
Modified Christiansen and Lysgaard instances

No.	Type	$\|V\|$	Q	$[l,u]$
mCL-1	A-32-5	32	70	[0.51,0.70]
mCL-2	A-33-5	33	70	[0.51,0.70]
mCL-3	A-34-5	34	70	[0.51,0.70]
mCL-4	A-36-5	36	70	[0.51,0.70]
mCL-5	A-37-5	37	70	[0.51,0.70]
mCL-6	A-38-5	38	70	[0.51,0.70]
mCL-7	A-39-5	39	70	[0.51,0.70]
mCL-8	A-44-6	44	70	[0.51,0.70]
mCL-9	A-45-6	45	70	[0.51,0.70]
mCL-10	A-45-7	45	70	[0.51,0.70]
mCL-11	A-46-7	46	70	[0.51,0.70]
mCL-12	A-48-7	48	70	[0.51,0.70]
mCL-13	A-53-7	53	70	[0.51,0.70]
mCL-14	A-54-7	54	70	[0.51,0.70]
mCL-15	P-50-7	50	70	[0.51,0.70]
mCL-16	P-51-10	51	70	[0.51,0.70]
mCL-17	P-60-10	60	70	[0.51,0.70]
mCL-18	P-55-7	55	70	[0.51,0.70]

5.2.2. *Computational results*

Table 7 shows the comparison between "Unsplit-VRPSD" and "Search heuristic" and the comparison between "Construction heuristic" and "Search heuristic". It clearly shows that the

TABLE 5.
Performance of the insertion and removal operators measured by the mean value of the final weight in the modified Solomon instances

	Insertion				Removal			
$[l,u]$	SI	GI	RI	DFSI	SR	DWR	RWR	RR
[0.01,0.25]	0.080686	4.874835	0.723686	0.683987	4.135273	2.763767	3.467894	3.701768
[0.26,0.50]	0.287942	3.736304	3.764059	0.546953	2.723432	2.794918	4.518715	1.688778
[0.51,0.70]	1.144012	1.17171	1.977096	0.741155	1.542572	2.071047	0.884178	0.705601
–	0.115059	4.180278	1.328321	0.289084	3.591041	2.443624	2.590899	2.530475
Average	0.406925	3.490783	1.948291	0.565295	2.998080	2.518339	2.865422	2.156656

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

TABLE 7.
Comparisons on the modified Christiansen and Lysgaard instances

		Compared-VRPSD			Search heuristic			
No.	$\|V\|$	# Routes	Cost	Seconds	# Routes	Cost	Seconds	Imp(%)
mCL-1	32	16	3723.537	206.359	17	3618.770	550.031	2.81
mCL-2	33	16	2687.339	288.313	17	2568.234	491.969	4.43
mCL-3	34	17	3143.310	486.688	16	3077.386	929.688	2.10
mCL-4	36	18	3851.890	207.797	20	3619.477	702.797	6.03
mCL-5	37	18	2733.035	351.969	18	2525.722	1185.078	7.59
mCL-6	38	19	3102.566	550.547	19	2926.980	550.078	5.66
mCL-7	39	19	3638.229	355.641	18	3419.005	1206.703	6.03
mCL-8	44	22	4205.458	466.453	22	3992.892	1300.547	5.05
mCL-9	45	22	4405.244	313.219	23	4050.652	762.078	8.05
mCL-10	45	22	5160.982	435.813	23	4883.583	771.656	5.37
mCL-11	46	23	4032.453	394.000	23	3937.417	1799.516	2.36
mCL-12	48	24	5369.738	823.453	25	5009.487	874.969	6.71
mCL-13	53	26	4956.482	524.750	28	4725.685	1277.703	4.66
mCL-14	54	27	5562.702	582.547	24	5262.227	2482.469	5.40
mCL-15	50	25	2307.653	409.609	25	2196.238	1501.234	4.83
mCL-16	51	25	2349.025	953.031	23	2235.803	2854.156	4.82
mCL-17	60	30	2824.853	634.547	30	2734.645	2422.734	3.19
mCL-18	55	27	2463.674	763.641	28	2403.593	1115.109	2.44
Average		**22**	**3695.454**	**486.021**	**22.167**	**3510.433**	**1265.473**	**4.86**

average number of routes and the average cost of the solutions obtained by "Search heuristic" are less than in the solutions obtained by "Unsplit-VRPSD". Because the solution search space becomes larger for the case of split deliveries, the average CPU time used by "Search heuristic" is larger than for "Unsplit-VRPSD". The average percentage improvement in "Cost" obtained by our heuristic, compared with the unsplit VRPSD, is 4.86%.

Table 8 provides the total visit number of vertices in the solutions of Table 7. We can see that the total number of visits in each instance is larger than the number of vertices. This is due to the split deliveries which explain the improvement "Imp(%)" in Table 7. Again, the best split delivery solutions contain more routes.

Table 9 shows the performance of the insertion and removal operators on the modified Christiansen and Lysgaard instances.

TABLE 8.
Total number of vertex visits in the modified Christiansen and Lysgaard instances

No.	$\|V\|$	# visits	No.	$\|V\|$	# visits	No.	$\|V\|$	# visits
mCL-1	32	36	mCL-7	39	42	mCL-13	53	65
mCL-2	33	39	mCL-8	44	51	mCL-14	54	61
mCL-3	34	36	mCL-9	45	52	mCL-15	50	56
mCL-4	36	42	mCL-10	45	53	mCL-16	51	58
mCL-5	37	43	mCL-11	46	49	mCL-17	60	63
mCL-6	38	44	mCL-12	48	57	mCL-18	55	62

TABLE 9.
Performance of the insertion and removal operators measured by the mean value of the final weight in the modified Christiansen and Lysgaard instances

Insertion				Removal			
SI	GI	RI	DFSI	SR	DWR	RWR	RR
1.060518	1.319132	2.767663	0.776141	1.543397	2.776835	0.816260	0.867634

ISSN 0315-5986 | EISSN 1916-0615 DOI 10.3138/infor.50.2.059

It reports the weights associated with each operator, which is obtained as the mean value of the final weights used to get the results of Tables 7. In our tests, the best insertion operators were GI and RI, whereas the best removal operators were SR and DWR.

6. CONCLUSION

We have proposed and implemented a new paired vehicles recourse policy with split delivery for the Vehicle Routing Problem with Stochastic Demands and Split Deliveries. An adaptive large neighborhood search heuristic in which some simple heuristics compete to destroy and repair the current solution was developed. Modified Solomon instances and modified Christiansen and Lysgaard instances were both used in the experiments. Our experimental results show that using split deliveries results in significant savings in some specific ranges of the ratio between the vertex demand and vehicle capacity, and yields a similar performance otherwise.

ACKNOWLEDGEMENTS

This work was partly supported by the Canadian Natural Sciences and Engineering Research Council under grant 39682-10, by the Elite Plan Program of the Chinese National University of Defense Technology and by the Chinese National Natural Science Foundation under grant 70971132. This support is gratefully acknowledged. Thanks are due to the Associate Editor and to the referees for their valuable comments.

REFERENCES

Ak, A. and Erera, A. L. (2007), "A paired-vehicle recourse strategy for the vehicle routing problem with stochastic demands", *Transportation Science*, 41: 222–237.

Archetti, C., Savelsbergh, M. W. P., and Speranza, M. G. (2006), "Worst-case analysis for split delivery vehicle routing problems", *Transportation Science*, 40: 226–234.

Archetti, C., Savelsbergh, M. W. P., and Speranza, M. G. (2008), "To split or not to split: That is the question", *Transportation Research Part E: Logistics and Transportation Review*, 44: 114–123.

Belenguer, J. M., Martinez, M. C., and Mota, E. (2000), "A lower bound for the split delivery vehicle routing problem", *Operations Research*, 48: 801–810.

Bertsimas, D. J. (1992), "A vehicle routing problem with stochastic demand", *Operations Research*, 40: 574–585.

Bertsimas, D. J., Chervi, P., and Peterson, M. (1995), "Computational approaches to stochastic vehicle routing problems", *Transportation Science*, 29: 342–352.

Bertsimas, D. J., Jaillet, P., and Odoni, A. R. (1990), "A priori optimization", *Operations Research*, 38: 1019–1033.

Bianchi, L., Birattari, M., Chiarandini, M., Manfrin, M., Mastrolilli, M., Paquete, L., Rossi-Doria, O., and Schiavinotto, T. (2004), "Metaheuristics for the Vehicle Routing Problem with Stochastic Demands", *Parallel Problem Solving from Nature-PPSN VIII, Lecture Notes in Computer Science*, Vol. 3242. Springer-Verlag, Berlin, pp. 450–459.

Christiansen, C. H. and Lysgaard, J. (2007), "A branch-and-price algorithm for the capacitated vehicle routing problem with stochastic demands", *Operations Research Letters*, 35: 773–781.

Cordeau, J.-F., Laporte, G., Savelsbergh, M. W. P., and Vigo, D. (2007), "Vehicle routing", In Barnhart, C. and Laporte, G. (eds.), *Transportation Handbooks in Operations Research and Management Science*, Vol. 14, North-Holland, Amsterdam, pp. 367–428.

Chepuri, K. and Homem-de Mello, T. (2005), "Solving the vehicle routing problem with stochastic demands using the cross-entropy method", *Annals of Operations Research*, 134: 153–181.

Dror, M., Laporte, G., and Trudeau, P. (1989), "Vehicle routing with stochastic demands: Properties and solution frameworks", *Transportation Science*, 23: 166–176.

Dror, M., Laporte, G., and Trudeau, P. (1994), "Vehicle routing with split deliveries", *Discrete Applied Mathematics*, 50: 239–254.

Dror, M. and Trudeau, P. (1986), "Stochastic vehicle routing with modified savings algorithm", *European Journal of Operational Research*, 23: 228–235.

Dror, M. and Trudeau, P. (1989), "Savings by split delivery routing", *Transportation Science*, 23: 141–145.

Dror, M. and Trudeau, P. (1990), "Split delivery routing", *Naval Research Logistics*, 37: 383–402.

Dror, M. (1993), "Modeling vehicle routing with uncertain demands as a stochastic program: Properties of the corresponding solution", *European Journal of Operational Research*, 64: 432–441.

Dueck, G. (1993), "New optimization heuristics: The great deluge algorithm and the record-to-record travel", *Journal of Computational Physics*, 104: 86–92.

Gendreau, M., Laporte, G., and Séguin, R. (1995), "An exact algorithm for the vehicle routing problem with stochastic demands and customers", *Transportation Science*, 29: 143–155.

Gendreau, M., Laporte, G., and Séguin, R. (1996), "A tabu search heuristic for the vehicle routing problem with stochastic demands and customers", *Operations Research*, 44: 469–477.

Goodson, J., Ohlmann, J., and Thomas, B. (2012), "Cyclic-order neighborhoods with application to the vehicle routing problem with stochastic demand", *European Journal of Operational Research*, 217: 312–323.

Ho, S. C. and Haugland, D. (2004), "A tabu search heuristic for the vehicle routing problem with time windows and split deliveries", *Computers & Operations Research*, 31: 1947–1964.

Laporte, G. and Louveaux, F. V. (1993), "The integer L-shaped method for stochastic integer programs with complete recourse", *Operations Research Letters*, 13: 133–142.

Laporte, G., Louveaux, F. V., and Van hamme, L. (2002), "An integer L-shaped algorithm for the capacitated vehicle routing problem with stochastic demands", *Operations Research*, 50: 415–423.

Laporte, G., Musmanno, R., and Vocaturo, F. (2010), "An adaptive large neighbourhood search heuristic for the capacitated arc routing problem with stochastic demands", *Transportation Science*, 44: 125–135.

Lei, H., Laporte, G., and Guo, B. (2011), "The capacitated vehicle routing problem with stochastic demands and time windows", *Computers & Operations Research*, 38: 1775–1783.

DOI 10.3138/infor.50.2.059

Mendoza, J. E., Castanier, B., Guéret, C., Medaglia, A. L., and Velasco, N. (2010), "A memetic algorithm for the multi-compartment vehicle routing problem with stochastic demands", *Computers & Operations Research*, 37: 1886–1898.

Novoa, C., Berger, R., Linderoth, J., and Storer, R. (2006), A set-partitioning-based model for the stochastic vehicle routing problem, Technical Report 06T-008, Lehigh University.

Nowak, M., Ergun, O., and White, C. C. (2008), "Pickup and delivery with split loads", *Transportation Science*, 42: 32–43.

Potvin, J.-Y. and Rousseau, J.-M. (1993), "A parallel route building algorithm for the vehicle routing and scheduling problem with time windows", *European Journal of Operational Research*, 66: 331–340.

Ropke, S. and Pisinger, D. (2006), "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows", *Transportation Science*, 40: 455–472.

Shaw, P. (1997), A new local search algorithm providing high quality solutions to vehicle routing problems. Technical report, Department of Computer Science, University of Strathclyde, Glasgow.

Shaw, P. (1998), "Using constraint programming and local search methods to solve vehicle routing problems", *Principles and Practice of Constraint Programming - CP98,* Lecture Notes in Computer Science, Vol. 1520. Springer-Verlag, Berlin, pp. 417–431.

Solomon, M. M. (1987), "Algorithms for the vehicle routing and scheduling problems with time window constraints", *Operations Research*, 35: 254–265.

Tan, K. C., Cheong, C. Y., and Goh, C. K. (2007), "Solving multi-objective vehicle routing problem with stochastic demand via evolutionary computation", *European Journal of Operational Research*, 177: 813–839.

Yang, W. H., Mathur, K., and Ballou, R. H. (2000), "Stochastic vehicle routing problem with restocking", *Transportation Science*, 34: 99–112.

ISSN 0315-5986 | EISSN 1916-0615
DOI 10.3138/infor.50.2.059